Transactional Outbox

Season: 4 · Enterprise · Category: Enterprise · Level: Intermediate

Intent

- Atomically store message with business row; publish later.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Outbox tray: write letter, mailman picks up later.

When to Use

- Microservices / modular monoliths.
- The design force named in Intent shows up more than once.

When Not to Use

- Greenfield CRUD without distribution.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.enterprise.outbox;

import java.util.ArrayList;
import java.util.List;

record OutboxMessage(String id, String topic, String payload, boolean published) {
    OutboxMessage markPublished() { return new OutboxMessage(id, topic, payload, true); }
}

/** Transactional outbox — persist business row + message atomically, publish async. */
public final class Outbox {
    private final List<String> orders = new ArrayList<>();
    private final List<OutboxMessage> messages = new ArrayList<>();

    public synchronized void placeOrder(String orderId, String payload) {
        orders.add(orderId);
        messages.add(new OutboxMessage(orderId, "orders", payload, false));
    }

    public synchronized List<OutboxMessage> unpublished() {
        return messages.stream().filter(m -> !m.published()).toList();
    }

    public synchronized void publishDue() {
        for (int i = 0; i < messages.size(); i++) {
            var m = messages.get(i);
            if (!m.published()) messages.set(i, m.markPublished());
        }
```

Classic Implementation (2)

```
    }

    public List<String> orders() { return List.copyOf(orders); }
}
```

Modern Java 25 / Functional

- Sealed events + pattern matching for projections.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: enterprise/outbox/
```

Pitfalls

- Keep demos minimal — production needs idempotency and observability.
- Saga compensations must be careful.

Interview Q&A

- Q: Production concern for Transactional Outbox? A: Idempotency, observability, and failure modes.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Atomically store message with business row; publish later.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Outbox tray: write letter, mailman picks up later. [0:40] Intent — State the design force in one sen...